Vietnam National University, Ho Chi Minh City
University of Technology
Faculty of Computer Science and Engineering



DISCRETE STRUCTURE
(CO1007)

Assignment

A ALGORITHMS*

(November 4th, 2026)

Instructor(s): Nguyễn An Khương, *CSE-HCMUT*
Lê Hồng Trang, *CSE-HCMUT*
Trần Tuấn Anh,*CSE-HCMUT*
Phạm An Vinh,*CSE-HCMUT*
Mai Xuân Toàn, *CSE-HCMUT*
Trần Hồng Tài, *CSE-HCMUT*

Ho Chi Minh City, November 2026

Table of Contents

1 Introduction **1**
1.1 Graph Theory . . . 1
1.2 Path in a Graph . . . 1
1.3 Pathfinding Algorithms . . . 1
1.4 The Shortest Path Problem . . . 2
1.5 A* Algorithm . . . 2

2 Project Requirements **3**
2.1 Programming Tasks . . . 3
2.2 Function Naming for Auto Testing . . . 6
2.3 Common Data Structure Requirement . . . 6

3 VERSION INFO & QUESTIONS **7**
3.1 Version Info . . . 7
3.2 Guidelines for Asking Questions . . . 7
3.3 FAQ . . . 8

4 Instructions and Submission guideline **8**
4.1 Instruction . . . 8
4.2 Submission . . . 8
4.3 Cheating treatment . . . 9

1 Introduction

1.1 Graph Theory

Graph theory is a foundational area of discrete mathematics that studies the relationships between objects using vertices and edges. A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E, where each edge connects a pair of vertices. Graphs can be directed or undirected, weighted or unweighted, and may represent a wide range of systems such as networks, maps, or decision trees. In computer science, graphs are used to model data structures, optimize routes, and analyze connectivity. Directed graphs (digraphs) are particularly useful in pathfinding problems, where each edge has a direction and possibly a cost associated with traversal. Weighted graphs assign numerical values to edges, representing distances, time, or other metrics. Graph theory provides the mathematical framework for designing and analyzing algorithms that operate on these structures, including traversal, search, and optimization techniques. Understanding graph properties—such as connectivity, cycles, and shortest paths—is essential for solving complex problems in fields like artificial intelligence, logistics, and operations research. In this project, we use a directed weighted graph to represent a maze, where each cell is a node and each valid move is an edge.

1.2 Path in a Graph

A path in a graph is a sequence of vertices connected by edges, representing a route from one point to another. Formally, a path from vertex v_0 to v_n is a sequence $v_0, v_1, \ldots, v_n$ such that each pair (v_i, v_{i+1}) is an edge in the graph. The length of a path is typically defined as the number of edges it contains, while the cost of a path in a weighted graph is the sum of the weights of its edges. Paths can be simple (no repeated vertices), cyclic (starting and ending at the same vertex), or optimal (minimizing cost or length). In practical applications, paths are used to model navigation, communication, and decision-making processes. For example, in a maze represented as a graph, a path corresponds to a sequence of moves from the entrance to the exit. Identifying valid paths requires checking adjacency and avoiding obstacles or invalid transitions. In optimization problems, the goal is often to find the shortest or least-cost path among many possibilities. Algorithms like Dijkstra's and A* are designed to efficiently explore graphs and identify optimal paths based on specific criteria.

1.3 Pathfinding Algorithms

Pathfinding algorithms are computational techniques used to determine a viable or optimal route between two points in a graph. These algorithms play a crucial role in various domains such as robotics, logistics, computer games, and navigation systems. At their core, pathfinding methods explore the structure of a graph to identify a sequence of edges that connect a start node to a goal node, often while minimizing cost, distance, or time.

In weighted graphs, each edge carries a numerical value representing the cost of traversal. Pathfinding algorithms must account for these weights to avoid inefficient or costly routes. Some algorithms guarantee the shortest path, while others prioritize speed or simplicity, depending on the application. Common strategies include breadth-first search (BFS), depth-first search (DFS), and uniform cost search, each with its own strengths and limitations.

Effective pathfinding depends on how the graph is represented, how neighbors are evaluated, and how decisions are made during traversal. In maze-solving problems, for example, the graph is typically a grid, and the algorithm must navigate around walls, dead ends, and open paths.

The final output is a sequence of nodes forming a valid path from the start to the goal, often reconstructed by tracing back through visited nodes once the goal is reached.

1.4 The Shortest Path Problem

The shortest path problem is a fundamental question in graph theory and computer science, concerned with finding the minimum-cost path between two vertices in a graph. Given a weighted graph $G = (V, E)$, where each edge $(u, v) \in E$ has an associated non-negative weight $w(u, v)$, the goal is to determine a path from a source vertex s to a destination vertex t such that the sum of the edge weights along the path is minimized.

This problem has numerous real-world applications, including GPS navigation, network routing, transportation planning, and robotics. Depending on the graph's structure and constraints, different algorithms may be employed. For instance, Dijkstra's algorithm efficiently solves the single-source shortest path problem in graphs with non-negative weights, while the Bellman-Ford algorithm can handle graphs with negative edge weights. In cases where all edge weights are equal, a breadth-first search (BFS) can be used to find the shortest path in terms of the number of edges.

The shortest path problem also serves as the foundation for more advanced optimization techniques, such as those used in heuristic search, dynamic programming, and reinforcement learning. Understanding this problem is essential for designing intelligent systems that operate efficiently in complex environments.

1.5 A* Algorithm

The A* (A-star) algorithm is a widely used pathfinding and graph traversal technique in computer science and artificial intelligence. It is designed to find the shortest path from a starting node to a goal node in a weighted graph or grid-based environment.

A* combines the strengths of Dijkstra's algorithm and Greedy Best-First Search by using a cost function defined as:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the actual cost from the start node to the current node.
- $h(n)$ is the heuristic estimate of the cost from the current node to the goal.
- $f(n)$ is the total estimated cost of the cheapest solution through node n.

If the heuristic $h(n)$ is admissible (i.e., it never overestimates the true cost to reach the goal), A* is guaranteed to find the optimal path. The choice of a heuristic has a significant impact on the algorithm's efficiency and accuracy.

A* is commonly applied in:

- Maze and map navigation
- Puzzle solving (e.g., 8-puzzle)
- Robotics and autonomous movement
- Game AI and route planning

For example, the graph below illustrates a simple example of the A* pathfinding algorithm. Each node is labeled with its name, heuristic value $h(n)$, and total cost estimate $f(n) = g(n) + h(n)$. In this graph:

- Node **S** is the **start node**.
- Node **G** is the **goal node**.
- The optimal path is highlighted in red: **S** $\rightarrow$ **A** $\rightarrow$ **D** $\rightarrow$ **G**.

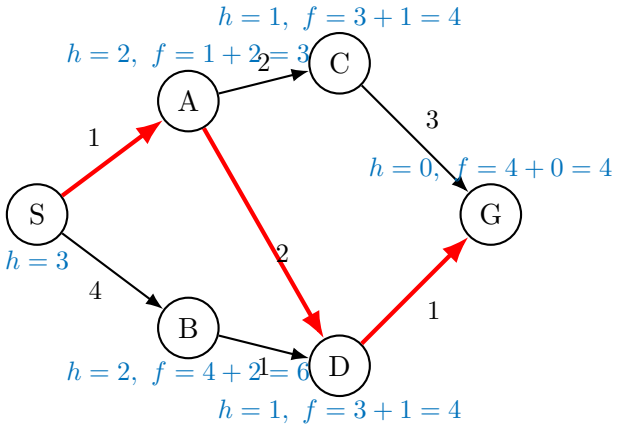


Fig. 1: *Example for A* algorithm*

In this project, students will implement A* in various contexts and explore its behavior under different heuristic strategies.

2 Project Requirements

2.1 Programming Tasks

Students are required to complete the 5 programming tasks described in this section, each involving the application of the A* search algorithm combined with concepts from discrete mathematics to solve a practical problem. All implementations must be written in **C++**, using only the default libraries provided by the LMS CodeRunner environment, which include: `<iostream>`, `<fstream>`, `<string>`, `<cmath>`, `<vector>`, and `<algorithm>`. Finally, students must submit the following:

- `PathNode.h` and `PathNode.cpp` for the `PathNode` struct and related functions.
- `Algo.h` and `Algo.cpp` containing all functions that solve the tasks.
- A `main.cpp` file that demonstrates the program's functionality using sample input and output.
- A report file that answers the report requirements for each task, named after your student ID (e.g., `24252628.pdf`).

1. **Mesh Network Packet Routing with Congestion Penalty (2.0 pts):** Given a communication mesh network represented by an adjacency matrix containing base latency $L(u, v) > 0$, implement the A* algorithm to find the optimal routing path between two network nodes. To account for node congestion, the cost of moving from node u to node v incorporates a penalty proportional to the degree (number of active connections) of target node v.

- **Input:** Weight matrix, start node, goal node.
- **Output:** A linked list containing the path of network nodes from start to goal, along with $f(n)$, $g(n)$, and $h(n)$ values at each step.
- **Cost Function:**
  - $g(n)$: Accumulated traversal latency from the source node to the current node, calculated as:
$$\text{Cost}(u \to v) = \text{weightMatrix}[u][v] + 0.1 \times \text{degree}(v)$$
  - $h(n)$: Estimated minimum routing latency remaining to reach the destination node.
  - $f(n) = g(n) + h(n)$
- **Report Requirement:** Explain how node degrees are computed from the adjacency matrix during path generation, how $h(n)$ is formulated, and provide a manual execution trace on a sample network.

2. **AUV Submarine Navigation in 3D Space (2.0 pts):** Given a set of underwater hydrographic survey stations defined by 3D coordinates (x, y, z) (where z represents depth below sea level) and a weighted edge matrix representing energy costs, implement A* to navigate an Autonomous Underwater Vehicle (AUV). Traversing against vertical water pressure adds a depth penalty.

   - **Input:** Weight matrix, 3D coordinates array `coords[100][3]`, start station, goal station, heuristic mode.
   - **Output:** A linked list containing the sequence of stations along the path, with $f(n)$, $g(n)$, and $h(n)$ values at each vertex.
   - **Cost Function:**
     - $g(n)$: Accumulated travel energy cost incorporating depth-difference adjustments:
$$g(n_{\text{new}}) = g(n_{\text{old}}) + w(u, v) \times \left(1 + \frac{|z_{\text{new}} - z_{\text{old}}|}{100}\right)$$
     - $h(n)$: Estimated 3D spatial distance from the current station to the target station.
     - $f(n) = g(n) + h(n)$
   - **Heuristic Modes (3D Spatial Metrics):**
     - Mode 1: 3D Manhattan distance — $h(n) = |x_n - x_g| + |y_n - y_g| + |z_n - z_g|$
     - Mode 2: 3D Euclidean distance — $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2 + (z_n - z_g)^2}$
     - Mode 3: 3D Chebyshev distance — $h(n) = \max(|x_n - x_g|, |y_n - y_g|, |z_n - z_g|)$
   - **Report Requirement:** Compare the execution performance and path optimality among the three 3D heuristics with concrete test cases.

3. **Probabilistic Pathfinding under Hazard Risk (2.0 pts):** Given a network of navigation points with physical distances $d(u, v)$ and associated failure/hazard probabilities $P_{\text{risk}}(u, v) \in [0, 1)$, implement A* to balance path length against risk exposure using expected values.

   - **Input:** Distance matrix, hazard risk probability matrix, coordinates list, start point, goal point, risk penalty factor λ, heuristic mode.

- **Output:** A linked list representing the selected route with evaluated $f(n)$, $g(n)$, $h(n)$, and accumulated safety probability P_{safe}.
- **Cost Function:**
  - $g(n)$: Expected traversal cost accumulated along the path:

$$g(n) = \sum_{(u,v)\in \text{path}} [d(u,v) + \lambda \cdot P_{\text{risk}}(u,v)]$$

  - $P_{\text{safe}}(\text{path})$: Combined path survival probability calculated via independent probability multiplication:

$$P_{\text{safe}}(\text{path}) = \prod_{(u,v)\in \text{path}} (1 - P_{\text{risk}}(u,v))$$

  - $h(n)$: Distance estimate to goal calculated via spatial metrics (Manhattan/Euclidean).
  - $f(n) = g(n) + h(n)$
- **Report Requirement:** Analyze the impact of the penalty parameter λ on path selection (short/risky vs. long/safe routes) with a worked mathematical example.

4. **Game Character Navigation with Multi-Terrain Obstacles (2.0 pts):** Given a game terrain represented as an $m \times n$ grid with multiple terrain types (0: flat ground, 2: swamp/sand, 1: obstacle), implement A* to find the motion path for a unit allowing 8-directional movement.

   - **Input:** Grid layout, start coordinate, goal coordinate, heuristic mode.
   - **Output:** A linked list containing direction action labels (`Up`, `Down-Right`, etc.) alongside $f(n)$, $g(n)$, and $h(n)$ metrics.
   - **Movement Rules & Cost Function:**
     - **Terrain Cost:** Moving into a flat cell (0) costs 1.0. Moving into a swamp cell (2) costs 2.0. Obstacle cells (1) cannot be entered.
     - **Diagonal Movement:** Base cost is multiplied by 1.5. **Diagonal Corner Cutting Rule:** To move diagonally from (x, y) to $(x \pm 1, y \pm 1)$, both adjacent cardinal cells must NOT be obstacles (1). If either is an obstacle, the diagonal move is invalid.
     - $g(n)$: Accumulated movement cost based on terrain and direction.
     - $h(n)$: Distance estimate calculated using Manhattan (Mode 1) or Chebyshev (Mode 2) heuristics.
     - $f(n) = g(n) + h(n)$
   - **Report Requirement:** Provide a step-by-step trace on a small grid showing directional expansion decisions, corner-cutting checks, and cost updates.

5. **VLSI PCB Wire Routing with Bending Penalty (2.0 pts):** Given a Printed Circuit Board (PCB) grid layout of size $m \times n$, construct a full weighted graph representation and apply A* to route a conductive trace from a component pin to an output pad.

   - **Input:** PCB grid layout, start coordinate, exit coordinate, empty weight matrix, heuristic mode.
   - **Output:** A linked list of the trace route with (f, g, h) values, and the populated graph weight matrix.

- **Cost Function:**
  - $g(n)$: Accumulated trace length. If the current movement changes direction compared to the previous step (e.g., from `Up` to `Right`), a wire bending penalty $B = 0.5$ is added to $g(n)$.
  - $h(n)$: Estimated metric to exit (Mode 1: Manhattan, Mode 2: Chebyshev).
  - $f(n) = g(n) + h(n)$
- **Report Requirement:** Describe the grid-to-graph transformation algorithm and how the previous direction state is maintained during the A* search to calculate bending penalties.

2.2 Function Naming for Auto Testing

To facilitate automated testing, each programming task must be implemented as a function with the following standardized name and signature. The auto tester will use these function names to validate correctness.

- **Task 1 — Mesh Network Packet Routing:**
```
PathNode* findMeshPath(double weightMatrix[100][100], int startNode, int goalNode);
```
- **Task 2 — AUV Submarine Navigation in 3D Space:**
```
PathNode* findAUVPath(double weightMatrix[100][100], int coords[100][3],
int startPoint, int goalPoint, int mode);
```
- **Task 3 — Probabilistic Pathfinding under Hazard Risk:**
```
PathNode* findProbabilisticPath(double distMatrix[100][100],
double riskMatrix[100][100], int coords[100][2],
int startPoint, int goalPoint, double lambda, int mode);
```
- **Task 4 — Game Character Navigation with Multi-Terrain Obstacles:**
```
PathNode* findGamePath(int mapGrid[100][100], int m, int n, int startX,
int startY, int goalX, int goalY, int mode);
```
- **Task 5 — VLSI PCB Wire Routing:**
```
PathNode* findPCBPath(int pcbGrid[100][100], int m, int n, int startX,
int startY, int exitX, int exitY, double weightMatrix[100][100], int mode);
```

Each function must return a pointer to the head of a linked list of `PathNode` objects, representing the path from start to goal. The list should include the direction (e.g., Up, Down, Left, Right, Up-Left, Up-Right, Down-Left, Down-Right) or coordinate name, along with the cost values ($f(n)$, $g(n)$, $h(n)$) for each step.

2.3 Common Data Structure Requirement

For all five problems described in this project, students must define and use a common linked list data structure to represent the solution path. Each node in the linked list must contain the following fields:

- **Name:** a `std::string` representing the name or label of the node (e.g., vertex name, coordinate).

- $f(n)$**:** the total estimated cost from the start node to the goal through this node.
- $g(n)$**:** the actual cost from the start node to this node.
- $h(n)$**:** the heuristic estimate from this node to the goal.
- **Next:** a pointer to the next node in the path.

Example (C++ struct):

```
struct PathNode {
    std::string name;
    double f;
    double g;
    double h;
    PathNode* next;
};
```

This linked list should be used to store and output the final path found by the A* algorithm in each problem, preserving the order from the start node to the goal node. The following `printPath` function will be included in the LMS test case system for scoring. Make sure your struct works with it and **don't include this function in your LMS code runner submission**.

```
void printPath(PathNode* head) {
    cout << "Solution Path:\n";
    while (head != nullptr) {
        cout << "Node: " << head->name
             << " | f: " << head->f
             << " | g: " << head->g
             << " | h: " << head->h << "\n";
        head = head->next;
    }
}
```

3 VERSION INFO & QUESTIONS

3.1 Version Info

- Change log on 04/11/2026:
  - None yet

3.2 Guidelines for Asking Questions

If you have any questions about the assignment, please use the official Google Form:
https://forms.gle/p9BHAE8i47ribWRQA

Before submitting a question, you must first check whether it has already been answered in:

- The FAQ section (below),

- Other sections of this assignment document (instructions, requirements, submission rules, etc.), or
- The shared google sheet: Questions Response Sheet

Important Penalty Rule: Submitting a question that has already been answered in this document, the FAQ, or the answer sheet will result in a penalty point.

3.3 FAQ

- None yet

4 Instructions and Submission guideline

Students have to follow the instructions and comply with the requirements below. *Lecturers do not solve any cases arising because students do not follow the instructions or do not comply with the requirements.*

4.1 Instruction

- **Deadline:** All submissions must be completed by December 6th, 2026. Late submissions will not be accepted.
- **Programming Language:** All tasks must be implemented in C++.
- **Clarity:** Answers must be written clearly and coherently in both code and report form.

4.2 Submission

To receive a score, students must complete **both** submission steps:

1. **Auto-testing code submission:** Submit your code to the designated auto-testcase runner on the LMS site. Each task has its own section in the submission interface. ***Note: The auto-testing submission portal will be opened later.***
   *Any cases that require considering priority, refer to these test cases for the answers.
2. **File submission:** Prepare all required files listed in Section 2.1. Place them into a folder named after your student ID (e.g., `2121323`). Compress this folder into a `.rar` file **(Don't compress into another type then rename as .rar)** with the same name (e.g., `2121323.rar`). Submit this compressed file to the LMS file submission portal.
   - The file submission portal will be available only on the final day of the deadline.
   - **Penalties:** Each of the following errors will result in a **-1 point deduction per occurrence**:
     - Missing required file
     - Incorrect file name
     - Incorrect folder name
     - Incorrect compression format (must be .rar)
     - Disorganized file structure

4.3 Cheating treatment

This project must be completed **individually**. Collaboration is limited to general discussion of concepts; all submitted code, reports, and explanations must be your own work. Any violation of these rules will be treated as academic misconduct under university regulations. A student will be considered to have committed cheating if:

- **Unusual similarity of submissions:** If two or more students submit work with significant overlap in code, structure, or written content, **all parties** will be considered to have cheated. Protect your work and do not share files or solutions.
- **Lack of understanding of submitted work:** If a student cannot explain the meaning, logic, or reasoning behind their own code or report when questioned, the submission will be treated as dishonest. You may consult textbooks, online resources, or discussions, but you must write and implement the ideas in your own words and code.
- **Unauthorized copying:** Directly copying solutions from peers, online sources, or AI tools without proper adaptation and understanding is prohibited.

Important: Students are responsible for safeguarding their work. Sharing your files or allowing others to copy your solutions is considered cheating, even if you were not the one who copied. All cases of cheating will be handled in accordance with the university's disciplinary regulations.